Sprint 4 · Reto «Herramientas internas» · en una página

> Versión corta para **trabajar con ella abierta**. La ficha completa (glosario, uso de la IA, errores frecuentes, guía paso a paso) sigue en Reto_Sprint_04.md; **qué hacer en cada misión**, en las tarjetas de misión; la plantilla de código, en `reto_herramientas/`.

El cliente. TurisData calcula cada semana sus cifras de reservas a mano. Quiere un programa que las calcule solo.
Qué entregáis (no cambia): un repositorio Git con `kpis.py`, `test_kpis.py`, `README.md` y un `salida.json` generado con el CSV real. **Al final del sprint se suben a `sprint-04/` del repositorio del equipo: 4 ficheros del equipo (esos cuatro) + 1 autoevaluación y coevaluación por persona** (`autoevaluacion_nombre.md`).
Tiempo en clase: martes ≈ 185 min de misiones (+ 40 min de tutoría para acabar la misión 3) · miércoles 150 min (misiones 4 y 5, README y entrega). Lo que no quepa, a la tutoría o a casa.

Tres niveles: el equipo decide hasta dónde llega

Nivel	Qué hay que tener	Nota*
🟢 **Mínimo** (todos los equipos)	`leer_reservas`, `tasa_cancelacion_por_canal` e `ingreso_medio_por_reserva` pasan sus pruebas · `python kpis.py reservas_turisdata.csv salida.json` crea el JSON (la ocupación y `main` con ayuda del docente) · mensaje claro si falta el fichero · README corto · 3 commits	2
🟡 **Estándar**	Todas las pruebas en verde · errores controlados (fichero inexistente, columna que falta, fila defectuosa descartada y contada) · docstrings y PEP 8 · README con la definición de cada KPI y **por qué la ocupación es solo un proxy** · declaración del uso de IA	3
🔴 **Extra**	Pruebas propias además de las dadas · `kpis_pandas.py` con los mismos resultados · argumento `--por-mes` · `@dataclass` para `Reserva`	4

*Orientativo; puntúa el docente con la rúbrica del curso (0–4 en cada criterio). El mínimo es una entrega válida y evaluable; la «Definition of Done» de la ficha completa describe el nivel **estándar**.

Seis misiones, en dos fases (parejas piloto y copiloto, que rotan en cada misión)

Misión	Qué se construye (con su paquete de ayuda: curso ★ y Taller)	Min	Hecha cuando…
Martes · fase 1: la lógica, con las reservas en memoria			
0 · Mirar	Entender `demo_cliente.py` (curso ★ `type()` y f-strings)	35	Respondéis las 5 preguntas de lectura
1 · Datos	`mision_1_datos.py` (curso ★ listas y dicts · Taller 1)	50	«✅ ¡Misión 1 superada!»
2 · Decidir	`mision_2_decidir.py` (curso ★ `if` y `for`)	50	«✅ ¡Misión 2 superada!»
3 · Funciones	`tasa_cancelacion_por_canal` e `ingreso_medio_por_reserva` (curso ★ funciones · Taller 2)	50 + 40	`python test_kpis.py 1` → 4 de 4
Miércoles · fase 2: ficheros, errores y entrega			
4 · Ficheros	`_convertir_fila` y `leer_reservas` (curso ★ excepciones y ficheros · Taller 3)	50 + 50	`python test_kpis.py 2` → 7 de 8
5 · Salida	`exportar_json`, `main`, romperlo a propósito, README y entrega (Taller 4)	25 + 25	`python test_kpis.py` → 12 de 12

Cada día termina con una kata individual (10 min, en el Taller): martes `noches_confirmadas`; miércoles `media_precio`.

Frases para el README (completad los huecos)

- «`kpis.py` lee `reservas_turisdata.csv` y calcula … KPIs. Se ejecuta con `python kpis.py … salida.json --capacidad …`.»
- «La ocupación es un **proxy**: contamos todas las noches en la fecha de … y suponemos … habitaciones, así que …»
- «Si falta el fichero, el programa muestra «…» y termina con código 1.»
- «Hemos usado IA para …; lo hemos revisado …; no hemos pegado datos personales.»

Antes de entregar

- **Archivos subidos en `sprint-04/`:** `kpis.py` · `test_kpis.py` · `README.md` · `salida.json` · `autoevaluacion_nombre.md` (**uno por persona**) = 4 + N ficheros
- ☐ `python test_kpis.py` termina sin fallos · ☐ `salida.json` creado con el CSV real · ☐ ≥ 3 commits con mensaje claro
- ☐ README con cómo ejecutar, cómo probar y los KPIs · ☐ Autoevaluación y coevaluación: **una por persona**, subida a `sprint-04/`